

Assignment on A* Search

In modern smart warehouses such as those used by companies like **Amazon**, autonomous robots are used to move goods efficiently from storage locations to delivery points. These robots must navigate through a **grid-based** warehouse while avoiding obstacles like shelves and restricted zones.

To achieve optimal navigation, the **A*** search algorithm is widely used to compute the shortest path between two locations.

You are given a 2D grid-based warehouse map. Each cell in the grid can represent one of the following:

Symbol	Meaning
R	Robot starting position
D	Delivery point (goal)
. (dot)	Free space (walkable)
#	Obstacle (shelves/walls, cannot be crossed)

The robot must move from **R to D** using the shortest possible path while avoiding obstacles. You must use the **A*** search algorithm to compute the optimal path.

Movement Rules:

- **Up**
- **Down**
- **Left**
- **Right**

Diagonal movement is **not allowed**. Each move has a **unit cost**.

Heuristic Functions:

1. Manhattan Distance Heuristic:

$$h(n) = |x_{\text{goal}} - x_{\text{current}}| + |y_{\text{goal}} - y_{\text{current}}|$$

Where:

- a. $(x_{\text{current}}, y_{\text{current}})$ is the current node
- b. $(x_{\text{goal}}, y_{\text{goal}})$ is the goal node

2. Euclidean Distance Heuristic:

$$h(n) = \text{sqrt}[(x_{\text{goal}} - x_{\text{current}})^2 + (y_{\text{goal}} - y_{\text{current}})^2]$$

which is basically the straight line distance between two nodes.

The evaluation function should be: $f(n) = g(n) + h(n)$

Where:

- $g(n)$ -> the cost from the start to the current node, and

- $h(n)$ -> heuristic estimate to the goal

Tie-breaking Rule:

If two nodes have identical $f(n)$ values:

1. Choose the node with a **smaller $h(n)$**
2. If still tied, choose the node with the **lexicographically smaller coordinate (For example, node (2,3) chosen over (3,2))**

Task Requirements

You must write a program that:

1. Reads a grid map as input.
2. Identifies the robot start position (R) and delivery point (D).
3. Applies the **A*** search algorithm.
4. Finds the **shortest path from R to D**.
5. Outputs:
 - The **length of the path**
 - The **sequence of coordinates representing the path**
 - The **final grid with the path marked** (Use * to mark the cells that belong to the computed path, excluding R and D).
 - The **number of expanded nodes for each of the heuristics used (A node is considered expanded if it gets selected by the algorithm, and its neighbors are explored)**

Input Format:

N M
Row1
Row2
 ...
RowN

Where: N-> number of rows and M-> number of columns. Each row contains M characters (S, G, ., #)

Output Format:

Your program must print:

1. Shortest Path Length (including the source and the destination node)
2. List of coordinates in the path
3. Grid with path marked
4. Number of nodes expanded for each of the heuristics

Coordinates must be printed as: (row, column). Indexing starts from 0.

Sample Input	Sample Output
5 6 R..#..	Shortest Path Length: 9 Path:

<pre> .#.#.. .#.... .###.D </pre>	<pre> (0,0)->(0,1)->(0,2)->(1,2)->(2,2)->(2,3)->(2,4)->(3,4) ->(3,5) Grid with Path: R**#.. .*#*.. .****. .###*D Nodes Expanded: Euclidean Distance: 10 Manhattan Distance: 8 </pre>
---	--

Basic Pseudocode for A* Search:

```

A_STAR(start, goal):

    open_list ← {start}
    closed_list ← empty

    while open_list is not empty:

        current ← node in open_list with lowest f(n)

        if current = goal:
            return path from start to goal

        move current from open_list to closed_list

        for each neighbor of current:

            if neighbor is not traversable or already in closed_list:
                skip

            compute g(n), h(n), f(n)

            if neighbor not in open_list or new path is better:
                update neighbor's cost and parent
                add neighbor to open_list

    return "No path found"

```

Some Important Instructions:

- Do not blindly copy codes from any AI platform (ChatGPT, Gemini)
- An in-class evaluation will be taken to test your understanding of the topic.
- **Submit the .py or .ipynb file only by the deadline.**